# CCPC C++ 赛场速查册

定制版 · A4 双栏 · 适合打印装订

> 使用方式 先看报错或需求关键词，再翻目录。代码以 C++17 为准；模板只在满足前提时使用。

| 我遇到的问题 | 先查 |
| --- | --- |
| lambda / sort 第三个参数 / 自定义比较 | 第 3 节 |
| 迭代器、二维 vector、unordered_map、auto | 第 5 节 |
| struct、this、const、运算符与堆比较 | 第 6 节 |
| 二分边界、first true / last true | 第 7 节 |
| 复杂度、递归内存、按 n/数值范围选算法 | 第 8 节 |
| 单调队列、DFS/BFS、差分、背包、构造 | 第 9–12 节 |
| CE / WA / TLE / RE / segmentation fault | 第 13 节 |
| 提交前最后检查 | 第 14 节 |

> 规则提醒 CCPC 不同场次对纸质资料、电子资料、设备与网络的规定可能不同。打印携带前必须以当届官方通知和领队要求为准。

根据你在 Luogu 项目与算法任务中的历史问题整理

版本 2026-09-27 · 已合并近期新对话

# 目录


1. 先判断问题属于哪一类 3

2. 比赛骨架、输入输出与调试 3

3. Lambda 与比较器 - 高频重点 3

4. STL 算法：排序、选择、查找 3

5. 容器、迭代器、字符串与 auto 4

6. struct、const、运算符与优先队列 4

7. 二分查找与二分答案 5

8. 类型、复杂度、复制、reserve 与重置 5

9. 栈、队列、并查集与常用结构 6

10. 图论：建图、遍历与最短路 6

11. 最小生成树、前缀和与窗口 7

12. DP、贪心与数论基础 7

13. CE / WA / TLE / RE 排查 7

14. 提交前 60 秒清单 8

# 1. 先判断问题属于哪一类

## 报错或现象 -> 第一检查点

| 现象 | 先检查 |
| --- | --- |
| CE 编译失败 | 括号/分号、类型、头文件、函数签名、比较器返回 bool |
| WA | 题意、下标、边界、溢出、输出格式、算法前提 |
| TLE | 先算最坏复杂度；再看复制/清空/排序；最后才看 I/O |
| RE / 段错误 | 输入是否成功、负下标、越界、空容器、递归深度 |
| 结果随机 | 未初始化变量、越界、悬空引用/迭代器 |
| 本地正常提交错 | freopen、调试输出、编译标准、未定义行为 |

## 模板使用前四问

- 前提满足吗：数组有序、判定单调、边权非负、图是 DAG？
- 状态/变量的中文含义是什么？
- 最坏复杂度包含所有测试组后仍可行吗？
- 答案落在边界时，输出的是哪一个变量？

# 2. 比赛骨架、输入输出与调试

## 干净提交模板

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int,int>;

void solve() {
    // input -> compute -> output
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // cin >> T;  // only if the statement says so
    while (T--) solve();
}
```

## 快速 I/O 的规则

- ios::sync_with_stdio(false)：关闭 C/C++ 流同步。之后不要混用 scanf/printf。
- cin.tie(nullptr)：输入前不再自动刷新 cout。交互题按要求手动 flush。
- 普通换行用 '\n'；endl 会额外刷新缓冲区。

## 本地调试但不污染提交

```
#ifdef LOCAL
#define debug(x) cerr << #x << " = " << (x) << '\n'
#else
#define debug(x) ((void)0)
#endif
```

> 你遇到过 in.txt 存在但运行目录不对，freopen 失败，cin 随后读取失败，未初始化的 n 又被用于创建 vector，最后表现为 segmentation fault。提交版不要依赖本地文件。

# 3. Lambda 与比较器 - 高频重点

## Lambda 基本形状

```
[capture](parameters) -> return_type {
    // body
}

auto add = [](int a, int b) { return a + b; };
int c = add(2, 3);
```

| 捕获 | 含义 |
| --- | --- |
| [] | 不使用外部局部变量 |
| [=] | 按值捕获外部变量，默认不可改原变量 |
| [&] | 按引用捕获，可修改原变量 |
| [x] | 只按值捕获 x |
| [&x;] | 只按引用捕获 x |
| [&, x] | 默认按引用，但 x 按值 |

## sort 的第三个参数填什么

形式：sort(first, last, comp)。comp(a,b) 返回 true 表示 a 应排在 b 前面。它必须是严格比较：相等时返回 false。

```
sort(a.begin(), a.end());                 // ascending
sort(a.begin(), a.end(), greater<int>()); // descending

sort(v.begin(), v.end(), [](const Item& a, const Item& b) {
    if (a.score != b.score) return a.score > b.score;
    return a.id < b.id;
});
```

> 严禁 比较器写 a.x <= b.x。相等元素互相都返回 true，会破坏 strict weak ordering，结果属于未定义行为。

## 多条件排序

```
// score descending, time ascending, id ascending
sort(v.begin(), v.end(), [](const Node& a, const Node& b) {
    if (a.score != b.score) return a.score > b.score;
    if (a.time != b.time) return a.time < b.time;
    return a.id < b.id;
});
```

## 使用外部变量

```
int pivot = 10;
sort(a.begin(), a.end(), [pivot](int x, int y) {
    return abs(x - pivot) < abs(y - pivot);
});
```

## 比较器自检

- comp(x,x) 必须为 false。
- 若 comp(a,b) 为 true，则 comp(b,a) 不能也为 true。
- 相等关键字必须继续比较下一关键字或返回 false。
- 引用参数通常写 const T&，避免复制且不允许修改元素。

# 4. STL 算法：排序、选择、查找

## 整段最大/最小与求和

```
int mx = *max_element(a.begin(), a.end());
int mn = *min_element(a.begin(), a.end());
ll sum = accumulate(a.begin(), a.end(), 0LL);
```

> 注意 std::max 只比较给定值；整段最大值用 max_element。空容器不能解引用返回的 end。

## 第 k 小：nth_element

```
// k is zero-based
nth_element(a.begin(), a.begin() + k, a.end());
cout << a[k] << '\n';
```

执行后 a[k] 是排序后应位于 k 的元素；左侧都不大于它，右侧都不小于它，但两侧内部不保证有序。平均 O(n)。

## 四个二分函数 - 区间必须有序

| 函数 | 结果 |
| --- | --- |
| binary_search | 是否存在，返回 bool |
| lower_bound | 第一个 >= x |
| upper_bound | 第一个 > x |
| equal_range | 等于 x 的半开区间 [L,R) |

```
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end() && *it == x) {
    int pos = int(it - a.begin());
}
int cnt = upper_bound(a.begin(), a.end(), x)
        - lower_bound(a.begin(), a.end(), x);
```

### 去重与排列

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());

do {
    // use current permutation
} while (next_permutation(a.begin(), a.end()));
```

## 5. 容器、迭代器、字符串与 auto

### begin/end 与访问

```
auto it = a.begin();
cout << *it;           // pointed element
cout << it - a.begin();// index, random-access iterator

// a.end() is one past the last element
if (it != a.end()) cout << *it;
```

end() 不是最后一个元素。空容器中 begin()==end()。vector 插入、删除或扩容后，旧迭代器可能失效。

### 范围 for 中有没有 & 的区别

```
for (int x : a) {}        // copy, cannot change a
for (int& x : a) cin >> x; // reference, changes a
for (const auto& x : a) {} // read-only reference
```

### 同时遍历下标和值

```
for (size_t i = 0; i < a.size(); ++i) {
    cout << i << ' ' << a[i] << '\n';
}

for (size_t i = 0; i < a.size(); ++i)
    for (size_t j = 0; j < a[i].size(); ++j)
        cout << i << ' ' << j << ' ' << a[i][j];
```

### 二维 vector 与固定两个元素

```
vector<vector<int>> a(n, vector<int>(2));
a[i] = {x, y};

// inner length is always 2: clearer and contiguous
vector<array<int,2>> b(n);
b[i] = {x, y};
```

原生数组声明后不能写 arr = {x,y} 整体赋值；固定长度优先用 array，动态长度用 vector。

### vector 删除

```
a.pop_back();                      // remove last
a.erase(a.begin() + i);            // remove index i
a.erase(a.begin() + l, a.begin()+r);// remove [l,r)

// remove all values equal to x
a.erase(remove(a.begin(), a.end(), x), a.end());
```

> 复杂度 vector 中间删除通常为 O(n)，因为后面的元素要前移。erase 返回删除区间之后的新迭代器。

### size_t、npos 与字符串查找

```
size_t pos = s.find("abc");
if (pos == string::npos) { /* not found */ }

size_t p = s.find_first_not_of('0');
if (p == string::npos) s = "0";
else s.erase(0, p);
```

size_t 是无符号大小类型。警惕 s.size()-1 在空字符串时下溢。倒序时可用 int，或写 for (size_t i=n; i--->0; )。

### auto 遇到已声明变量名

auto 只负责推导当前声明的类型，不会自动引用同名旧变量。同一作用域不能重复声明同名变量；内层作用域可遮蔽外层变量，但比赛中应避免。

```
auto x = 1;   // x is int
// auto x = 2; // error: redeclaration in same scope
{
    auto x = 2; // legal, but hides outer x
}
```

### 返回两个或多个值

```
pair<int,int> range() { return {l, r}; }
auto [l, r] = range();

tuple<int,int,ll> info() { return {u, v, w}; }
auto [u, v, w] = info();
```

### unordered_map 高频用法

```
unordered_map<int,int> mp;
++mp[x];                    // insert/update
if (auto it=mp.find(x); it!=mp.end()) cout << it->second;
for (const auto& [key,val] : mp) { /* unordered */ }
mp.erase(x); mp.reserve(n*2);
```

> 副作用 mp[x] 在 x 不存在时会自动插入默认值 0。只想查询时用 find；遍历顺序不确定。

### set 与 unordered_set

| 容器 | 查询/插入 | 顺序与能力 |
|---|---|---|
| set | O(log n) | 自动有序；支持 lower_bound |
| unordered_set | 平均 O(1)，最坏 O(n) | 无序；不支持 lower_bound |

```
set<pair<int,int>> seen;
if (a > b) swap(a, b);          // normalize
if (!seen.insert({a,b}).second)  // false: already existed
    continue;
```

既要平均 O(1) 查重又要保留首次进入顺序：用 unordered_set<T> seen + vector<T> order。哈希容器本身不保留插入顺序。

## 6. struct、const、运算符与优先队列

### 结构体、初始化与结构化绑定

```
struct Node { int cost, pos, extra; };
Node a{10, 2, 0};
Node b{10, 2};       // extra becomes 0
auto [c, p, e] = a;  // names must match fields
```

### 两种 const 含义

```
bool operator()(const Node& a, const Node& b) const {
    return a.cost > b.cost;
}
// const Node&: no copy, cannot modify the argument
// trailing const: this member function cannot modify *this
```

### this、*this、复制与引用

```
Node next = *this;  // copy current object
Node& ref = *this;   // alias current object
Node* ptr = this;    // pointer to current object

next.cost++; // original unchanged
ref.cost++;  // original changed
```

### 运算符重载不会自动补齐

```
bool operator<(const Node& o) const {
    return tie(cost, extra) < tie(o.cost, o.extra);
}
// In C++17, defining < does not generate >, ==, <=, >=.
```

运算符参数数量和含义是固定的。若一次操作还需要容量 V 等额外信息，优先写具名函数 nextState(w,V)，不要把并非数学加法的逻辑硬塞进 operator+。

### 成员函数需要外界变量

```
Node nextState(int w, int V) const; // pass as parameter

auto cmp = [V](const Node& a, const Node& b) {
    return score(a,V) > score(b,V);
};
priority_queue<Node, vector<Node>, decltype(cmp)> pq(cmp);
```

### 默认字典序

pair 和 tuple 默认先比较第 1 项；相等再比较第 2 项，以此类推。因此把最重要的排序关键字放在前面。

### 大根堆与小根堆

```
priority_queue<int> mx; // largest first

priority_queue<int, vector<int>, greater<int>> mn;

// Dijkstra: {distance, vertex}, smallest distance first
priority_queue<pii, vector<pii>, greater<pii>> pq;
```

### 自定义堆比较器

```
struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        // smaller dist has higher priority
        return a.dist > b.dist;
    }
};
priority_queue<Node, vector<Node>, Cmp> pq;
```

> 最容易混淆 sort 的 comp(a,b)==true 表示 a 放前面；priority_queue 的比较器返回 true 表示 a 的优先级更低。因此小根堆常写 a.key > b.key。

### tuple 放进堆

```
using State = tuple<ll,int,int>; // dist, vertex, extra
priority_queue<State, vector<State>, greater<State>> pq;
pq.push({0, s, 0});
auto [d, u, extra] = pq.top();
```

## 7. 二分查找与二分答案

### 精确查找 - 闭区间

```
int l = 0, r = n - 1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (a[mid] == x) return mid;
    if (a[mid] < x) l = mid + 1;
    else r = mid - 1;
}
return -1;
```

为什么必须 mid+1/mid-1：mid 已经检查过且不可能是答案；若只写 l=mid，区间可能不缩小并死循环。

### 第一个 true - false false true true

```
int l = L, r = R, ans = R + 1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) ans = mid, r = mid - 1;
    else l = mid + 1;
}
```

### 最后一个 true - true true false false

```
int l = L, r = R, ans = L - 1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) ans = mid, l = mid + 1;
    else r = mid - 1;
}
```

### 二分答案四句自检

- check(mid) 的完整中文含义是什么？
- mid 增大时，真假如何变化？
- 我要第一个 true 还是最后一个 true？
- 无解、全真、全假时 ans 应该是什么？

> 你遇到过 P1873/P2678 中的关键不是套框架，而是 check 的含义、终点处理、可删除次数和最后输出的边界变量。

## 8. 类型、复杂度、复制、reserve 与重置

### 为什么用了 long long 还会爆

```
// promote before multiplication
ll area = 1LL * w * h;

ll check(...);            // return type must also be ll
// receiver/comparison must also fit
ll need, got;
```

检查整条链：输入 -> 表达式 -> 累加 -> 返回类型 -> 接收 -> 比较 -> 输出。中间任何一步用 int 都可能先溢出。

### 最大值与 unsigned 陷阱

```
unsigned int UINF = numeric_limits<unsigned int>::max();
ll LINF = numeric_limits<ll>::max();

// unsigned never becomes negative
// 0u - 1 wraps to a very large value
```

需要表示 -1、做差或倒序下标时，优先使用有符号整数。INF 参与加法前先确认当前状态可达，并防止再次溢出。

### 大致操作量

| 规模 | 通常考虑 |
|---|---|
| n <= 20 | O(2^n) / 搜索 / 状压 |
| n <= 500 | O(n^3) 视时限 |
| n <= 3000 | O(n^2) 视常数 |
| n <= 1e5 | O(n log n) / O(n) |
| n <= 1e6 | 通常接近 O(n) |

不要把“1 秒等于固定多少次”当定理。语言、常数、缓存、容器和评测机都会影响；先用数量级排除明显不可行方案。

### 先看维度，不要只看算法名字

- n 很小（如 n<=20）、数值很大（如 ai<=1e9）：优先枚举/DFS，不能按总和开背包。
- 总和 S 小：O(nS) 背包可能合适；n<=40 且 S 巨大：考虑折半搜索 O(2^(n/2))。
- Windows 下 long 通常是 32 位；总和、表达式值和方案数优先检查是否需要 long long。

### 递归次数不等于递归内存

深度优先递归即使访问 2^n 个节点，任意时刻通常只保留一条根到叶路径。若深度为 n 且每层只存常数变量，额外栈空间为 O(n)，时间才是 O(2^n)。只有保存全部方案或复制大容器才会占指数级内存。

### O(nq) 不是快读能救的

- 每个询问是否完整扫描 n 个元素？
- 函数参数是否按值复制 vector/map/unordered_map？
- 是否每次 clear/fill 一个大数组？
- 能否预处理、缓存相同询问、离线排序或利用小值域？

### 引用、const 引用与复制

```
int f(vector<int> a);        // copies O(n)
int f(const vector<int>& a); // no copy, read-only
int f(vector<int>& a);       // no copy, can modify
```

### reserve、resize、clear、fill

| 操作 | 效果 |
|---|---|
| v.reserve(n) | 只预留容量，size 不变，不能访问 v[i] |
| v.resize(n) | size 变成 n，新增元素初始化 |
| v.clear() | size 变 0，capacity 通常保留 |
| fill(v.begin(),v.end(),0) | 保留 size，把现有元素置 0，O(n) |

unordered_map.reserve 预留桶以减少 rehash，但不是预先插入元素。循环中只改少量位置时，可记录 changed positions，仅恢复这些位置。

# 9. 栈、队列、并查集与常用结构

## stack / queue / deque

```
stack<int> st; st.push(x); st.top(); st.pop();
queue<int> q; q.push(x); q.front(); q.pop();
deque<int> dq;
dq.push_front(x); dq.push_back(x);
dq.front(); dq.back();
```

调用 top/front/back/pop 前先确认非空，否则是未定义行为。queue 为 FIFO：最早入队且未删除的是 front；size() 通常 O(1)，但没有随机访问。

## 并查集

```
struct DSU {
    vector<int> p, sz;
    DSU(int n): p(n+1), sz(n+1,1) { iota(p.begin(),p.end(),
0); }
    int find(int x) { return p[x]==x ? x : p[x]=find(p[x]);
 }
    bool unite(int a,int b) {
        a=find(a); b=find(b); if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a; sz[a]+=sz[b]; return true;
    }
};
```

## 单调栈：右侧第一个更大

```
vector<int> ans(n, -1), st;
for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.back()] < a[i]) {
        ans[st.back()] = i; st.pop_back();
    }
    st.push_back(i);
}
```

## 单调队列：滑动窗口最大值

```
deque<int> q; // store indices; values decrease
for (int i=0;i<n;++i) {
    while(!q.empty() && q.front() < i-k+1) q.pop_front();
    while(!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i);
    if(i>=k-1) cout << a[q.front()] << '\n';
}
```

队首始终是窗口最大值；每个下标最多进出一次，总计 O(n)。若 i 每轮严格 +1，过期检查用 if 也够；while 对跳步或变长窗口更稳健。求最小值时反转队尾比较。

# 10. 图论：建图、遍历与最短路

## 邻接表

```
vector<vector<int>> g(n+1);          // unweighted
vector<vector<pair<int,int>>> wg(n+1); // {to, weight}

g[u].push_back(v);       // directed u -> v
wg[u].push_back({v, w});
// undirected: also add reverse edge
```

无向图的邻接表通常要显式加入 u->v 和 v->u；这是同一条无向边的两份邻接记录，不是重复答案。

## DFS - 递归遍历，不修改原图

```
void dfs(int u, const vector<vector<int>>& g,
         vector<char>& vis) {
    vis[u]=1; cout << u << ' ';
    for(int v:g[u]) if(!vis[v]) dfs(v,g,vis);
}
```

不要在访问第一个邻点后 return；否则其余邻边永远不搜。大图递归深度最坏 O(n)，长链可能爆栈，可改显式 stack。

## 指定小编号优先：邻接表只排序一次

```
for(int u=1;u<=n;++u)
    sort(g[u].begin(), g[u].end());
// then reuse the same graph for DFS and BFS
```

总排序复杂度为 sum O(du log du) <= O(m log n)；稠密图最坏 O(n^2 log n)。静态图中 vector+一次排序通常比复制并反复 pop priority_queue 更快，也不会破坏原图。

## 邻接矩阵不会真的虚构边

```
const ll INF = (1LL<<60);
vector<vector<ll>> mat(n+1, vector<ll>(n+1, INF));
for (int i=1;i<=n;++i) mat[i][i]=0;

for (int v=1;v<=n;++v) {
    if (mat[u][v]==INF) continue; // no stored edge
    ll w=mat[u][v];
}
```

矩阵中的每个格子都存在，未存边的位置只是哨兵值。矩阵一行的元素是权值，不能写 for(auto [v,w]:mat[u])；邻接表元素才可拆成 {终点,权值}。

## BFS - 无权最短路

```
vector<int> dist(n+1, -1);
queue<int> q; dist[s]=0; q.push(s);
while(!q.empty()) {
    int u=q.front(); q.pop();
    for(int v:g[u]) if(dist[v]==-1) {
        dist[v]=dist[u]+1; q.push(v);
    }
}
```

BFS 用队列而非普通递归；发现邻点时立刻标记再入队，若等出队才标记，同一点可能被重复入队。DFS/BFS 的具体访问序取决于邻接点顺序。

## Dijkstra - 非负边

```
const ll INF = (1LL<<62);
vector<ll> d(n+1, INF);
priority_queue<pair<ll,int>, vector<pair<ll,int>>,
               greater<pair<ll,int>>> pq;
d[s]=0; pq.push({0,s});
while(!pq.empty()) {
    auto [du,u]=pq.top(); pq.pop();
    if(du!=d[u]) continue; // stale entry
    for(auto [v,w]:g[u]) if(du+w<d[v]) {
        d[v]=du+w; pq.push({d[v],v});
    }
}
```

堆负责每次取当前距离最小的状态；松弛负责尝试通过 u 改进 v。同一点可多次入堆，旧记录弹出时跳过。负边不能直接用 Dijkstra。

## 直接边、单源距离与任意两点

| 要查询 | 通常查看/计算 |
|---|---|
| x-y 是否有直接边 | 查 g[x] 或 mat[x][y] |
| 固定 s 到所有点 | 一次 Dijkstra/BFS，得到 dist[v] |
| 单次 x 到 y | 以 x 为起点，读取 dist[y] |
| 大量任意两点 | 按 n、m、q 选 Floyd/多次最短路/其他方法 |

## 多关键字最短路状态

```
using Cost = pair<int,int>; // primary, tie-breaker
const Cost BAD = {INT_MAX, INT_MAX};
vector<Cost> best(n+1, BAD);

// use identical ordering everywhere
if (candidate < best[v]) {
    best[v] = candidate;
    pq.push({candidate.first, candidate.second, v});
}
```

> 状态完整性 进入堆前要一次性更新所有状态字段。只更新 dist 却漏掉 packages/weight，会让节点出队时携带半初始化状态。

若边权不可拆分，“总重量最小”不一定等于“使用容器最少”。目标是包数时，可用 {包数，当前包重量} 作字典序状态；包数相同仍要用当前重量打破平局。

### 拓扑排序 - DAG

```
queue<int> q;
for(int i=1;i<=n;i++) if(indeg[i]==0) q.push(i);
vector<int> order;
while(!q.empty()) {
    int u=q.front(); q.pop(); order.push_back(u);
    for(int v:g[u]) if(--indeg[v]==0) q.push(v);
}
bool isDAG = (int)order.size()==n;
```

## 11. 最小生成树、前缀和与窗口

### Kruskal

```
sort(edges.begin(), edges.end(), [](auto& a, auto& b) {
    return a.w < b.w;
});
ll ans=0; int used=0;
for(auto e:edges) if(dsu.unite(e.u,e.v)) {
    ans += e.w; ++used;
}
if(used != n-1) { /* disconnected */ }
```

### 前缀和与差分

```
pre[0]=0;
for(int i=1;i<=n;i++) pre[i]=pre[i-1]+a[i];
ll sumLR = pre[r]-pre[l-1];

diff[l]+=x; diff[r+1]-=x;
for(int i=1;i<=n;i++) a[i]=a[i-1]+diff[i];
```

差分含义：diff[i]=a[i]-a[i-1]。区间 [l,r] 加 x 只改两个边界；r+1 要有哨兵空间或先判越界。最后对 diff 做前缀和还原。

### 最大子段：前缀 0 与非空约束

```
ll pre=0, minPre=0, ans=LLONG_MIN;
for (ll x:a) {
    pre += x;
    ans = max(ans, pre-minPre); // subtract an earlier pref
ix
    minPre = min(minPre, pre);
}
```

pre[0]=0 才能表示从第一个元素开始的区间。ans 不能初始化为 0，否则全负数组会错误地选择空子段。

### 滑动窗口思考顺序

- 窗口 [l,r] 内维护的性质是什么？
- 加入 a[r] 后，什么时候不合法？
- 不合法时如何移动 l 并撤销贡献？
- 每个指针只前进，通常总复杂度 O(n)。

> 正确性速证 维护不变量；说明每种比较结果下移动哪个端点不会漏解；最后证明指针只前进、循环一定结束。窗口单调性通常依赖元素为正或其他题目条件。

## 12. DP、贪心与数论基础

### 0/1 背包 - 每件最多一次

```
vector<ll> dp(V+1, 0);
for(auto [w,val]:items)
    for(int j=V;j>=w;--j)
        dp[j]=max(dp[j],dp[j-w]+val);
```

倒序保证 dp[j-w] 仍是加入当前物品之前的状态。正序会在同一轮重复使用当前物品。

### 0/1 背包计数：平分与符号选择

```
vector<long long> ways(target+1);
ways[0]=1;
for(int w:weight)
    for(int s=target;s>=w;--s)
        ways[s] += ways[s-w];
```

所有物品分到两边且要求平衡：总和为奇数则无解，否则统计子集和 total/2。相同重量的不同物品仍是不同选择；左右镜像是否算同一种必须按题意判断。

### 完全背包 - 每件无限次

```
for(auto [w,val]:items)
    for(int j=w;j<=V;++j)
        dp[j]=max(dp[j],dp[j-w]+val);
```

### DP 五问

状态含义？初值？转移来源？计算顺序？答案位置？如果说不完整，先别写模板。

### 快速幂

```
ll qpow(ll a,ll b,ll mod) {
    ll r=1%mod; a%=mod;
    while(b) {
        if(b&1) r=(__int128)r*a%mod;
        a=(__int128)a*a%mod; b>>=1;
    }
    return r;
}
```

### gcd / lcm

```
ll g = gcd(a,b);
ll l = a / g * b; // divide first; still check overflow
```

### 构造题的固定流程

- 先控制最难的性质：gcd/lcm
  常制造相等、整除、倍数；异或按位；奇偶按 mod 2/mod 4。
- 主动降低自由度：令两变量相等、一个为 1、一个是另一个的倍数，再用剩余量补和。
- 用小数据发现分类，再证明覆盖所有余数情况；最后逐项检查整数、正数、总和和特殊条件。

### 贪心自检

必须能回答：为什么当前选择不会使最优答案变差？能否把任一最优解交换成包含当前选择的最优解？只凭“看起来最优”不够。

## 13. CE / WA / TLE / RE 排查

### CE：从第一条错误开始

- expected ';'：先看上一行括号和分号。
- invalid types 'int[int]'：把普通 int 当数组/容器下标访问。
- must have pointer type：对非指针/非迭代器使用 * 或 ->。
- no matching function：参数类型、const、引用或模板类型不匹配。
- 比较器必须可调用且返回可转为 bool 的值。

### WA：最小反例优先

- 重读：求恰好、至多、至少，还是第一个/最后一个？
- 测 n=0/1、全相等、严格有序、答案在两端、无解。
- 检查 0/1 下标混用、r+1/l-1、输出空格和换行。
- 区间关系先标准化端点再去重；重复输入不应重复施加同一约束。
- 检查 long long 整条传播链和取模时机。
- 二分结束后确认输出 ans/l/r 中哪一个。

### TLE：顺序不能反

- 先写最坏复杂度，包括 T、n、q 的乘积。
- 查每次询问中的完整扫描、容器复制、fill、sort。
- 查递归区间是否缩小；手写快排是否退化。
- 删除大规模 cout/cerr。
- 最后再做快速 I/O 和常数优化。

### RE / segmentation fault

- cin 是否成功？变量是否初始化？
- 下标是否为负，或 >= size？
- vector<T> a(n) 的最后合法下标是 n-1；写 a[n] 已越界，错误可能到程序结束才显现。
- 是否解引用 end()、访问空栈/空队列？
- vector 修改后是否继续用旧迭代器？

- 递归是否过深？局部数组是否过大？
- freopen 查找的是运行目录，不是源码目录。

### 运算符与优先级陷阱

```
if ((len & 1) == 0) { /* even */ }
// &  : bitwise AND
// && : logical AND with short-circuit
```

不要依赖记忆猜混合表达式的结合方式；位运算参与比较时主动加括号。空栈检查应放在 && 左侧，利用短路避免执行 top()。

## 14. 提交前 60 秒清单

### 环境与输出

- [ ] C++17；没有本地绝对路径；没有无条件 freopen。
- [ ] 没有调试输出、样例编号、数组回显。
- [ ] 多测 T 与题面一致；没有凭习惯多读一次。
- [ ] 关闭同步后没有混用 scanf/printf。

### 类型与容器

- [ ] 总和、乘积、路径、方案数是否需要 long long？
- [ ] 乘法前是否用 1LL 或更宽类型提升？
- [ ] 函数返回类型和接收类型同样足够宽？
- [ ] end() 未被解引用；空容器未 top/front/back。
- [ ] reserve 后没有误以为 size 已变大。

### 算法与边界

- [ ] 最坏复杂度含所有测试组仍可行。
- [ ] 前提成立：有序/单调/非负边/DAG。
- [ ] 算法维度与约束匹配：n 小就枚举，和小才按和做 DP。
- [ ] 二分每轮严格缩小，答案边界明确。
- [ ] 测过最小、最大、全相等、无解和边界答案。
- [ ] 有向/无向边添加正确；不可达输出符合题意。

### 团队复核口令

提交前向队友说：算法、复杂度、正确性关键、未验证边界、输出格式。让未编码的队友用 30 秒查题意与边界。

> 最后原则 模板不是答案。先确认含义和前提，再复制代码；连续两次同类错误后停止盲交，重新手算最小反例。